

Jenkins CI/CD Lab — 5 Beginner Pipeline Projects (Windows Agent)

Course: ISWE406L — Agile Development Process and DevOps All five projects share **Stage 1: Checkout from GitHub**. Stages 2–3 differ per project.

Prerequisites (set up once, before class)

1. **Jenkins** installed and running on a Windows machine, with the **Git plugin** installed.
 2. **Python** installed on the Jenkins agent, added to PATH (verify with `python --version` in cmd).
 3. **flake8** and **pytest** pre-installed on the agent so class time isn't lost to installs:

```
pip install flake8 pytest
```
 4. Each student has a **GitHub account** and a **public repository** for their project.
 5. In Jenkins: **New Item** → **Pipeline**, and either:
 - Paste the Jenkinsfile directly into the **Pipeline script** box, or
 - Select **Pipeline script from SCM** and point it at the student's GitHub repo (if they commit the Jenkinsfile to the repo root).
-

PROJECT 1: Build & Test Pipeline

Concept taught: basic CI — install dependencies, then run automated tests.

Files to create in the GitHub repo

`app.py`

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

`test_app.py`

```
from app import add, subtract  
  
def test_add():  
    assert add(2, 3) == 5  
  
def test_subtract():  
    assert subtract(5, 3) == 2
```

`requirements.txt`

`pytest`

`Jenkinsfile`

```
pipeline {  
    agent any
```

```

stages {
    stage('Checkout') {
        steps {
            git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
        }
    }

    stage('Install Dependencies') {
        steps {
            bat 'pip install -r requirements.txt'
        }
    }

    stage('Run Unit Tests') {
        steps {
            bat 'pytest test_app.py'
        }
    }
}

```

Execution procedure

1. Create a new GitHub repo, add `app.py`, `test_app.py`, `requirements.txt`, and `Jenkinsfile` to it, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo (or paste the Jenkinsfile directly).
3. Click **Build Now**.
4. Open **Console Output** — confirm all 3 stages run in order and both tests pass (2 **passed**).
5. **Try it broken:** change `add` to return the wrong value, push, and rebuild — show that the “Run Unit Tests” stage now fails and the pipeline stops there.

PROJECT 2: List Utilities — Build & Test Pipeline (with Parametrized Tests)

Concept taught: still the Build & Test pattern from Project 1, but introduces **parametrized tests** — a pytest feature that runs the same test logic against several different inputs, instead of writing a separate test function for each case. This is the natural next step once students are comfortable with basic `assert` tests.

Files to create in the GitHub repo

`app.py`

```

def find_max(numbers):
    return max(numbers)

def count_evens(numbers):
    return len([n for n in numbers if n % 2 == 0])

```

`test_app.py`

```

import pytest
from app import find_max, count_evens

@pytest.mark.parametrize("numbers, expected", [
    ([1, 5, 3], 5),

```

```

    ([-10, -2, -7], -2),
    ([4, 4, 4], 4),
])
def test_find_max(numbers, expected):
    assert find_max(numbers) == expected

@pytest.mark.parametrize("numbers, expected", [
    ([1, 2, 3, 4], 2),
    ([1, 3, 5], 0),
    ([2, 4, 6, 8], 4),
])
def test_count_evens(numbers, expected):
    assert count_evens(numbers) == expected

```

requirements.txt

pytest

Jenkinsfile

```

pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Install Dependencies') {
            steps {
                bat 'pip install -r requirements.txt'
            }
        }

        stage('Run Unit Tests') {
            steps {
                bat 'pytest test_app.py -v'
            }
        }
    }
}

```

Note the `-v` (verbose) flag — with parametrized tests, this is worth adding so the console output lists every individual input case that ran (e.g. `test_find_max[numbers0-5]`), rather than collapsing them into one line.

Execution procedure

1. Create a new GitHub repo, add `app.py`, `test_app.py`, `requirements.txt`, and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Open **Console Output** — point out that pytest ran **6 tests total**, even though only 2 test functions were written — each parametrized case counts as its own test.

5. **Try it broken:** add one more case to the `find_max` parametrize list with a wrong expected value (e.g. `([1, 5, 3], 999)`), push, and rebuild — show that only that specific case fails, while the others still pass. This is a good moment to explain that parametrization gives fine-grained, per-input feedback.

PROJECT 3: Manual Approval Gate — Deploy Pipeline

Concept taught: this one is *not* the Build & Test pattern — it introduces a completely different, very common real-world CI/CD idea: a **manual approval gate**. Many pipelines shouldn't deploy automatically the moment code is checked in — someone should look and click “yes, go ahead” first. Jenkins supports this natively with the `input` step, which literally pauses the pipeline mid-run and waits for a human.

Files to create in the GitHub repo

`app.py`

```
print("Deploying application version 1.0...")
print("Deployment complete.")
```

`Jenkinsfile`

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'
            }
        }

        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                echo 'Build successful: app.py compiled with no syntax errors'
            }
        }

        stage('Deploy') {
            steps {
                input message: 'Approve deployment to production?', ok: 'Deploy'
                bat 'python app.py'
            }
        }
    }
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, then push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**.
4. Watch the pipeline reach the **Deploy** stage and then simply **stop moving** — no error, no crash, it's just waiting. In the Jenkins UI (either the build's console output or the pipeline visualization), a prompt appears: “*Approve deployment to production?*” with **Deploy** and **Abort** buttons.

5. Click **Deploy** — the pipeline resumes, runs `python app.py`, and finishes successfully.
6. **Try the other path:** run the build again, and this time click **Abort** instead. Show that the pipeline stops there and is marked as **aborted**, and `app.py` never actually runs — nothing gets “deployed” unless a human explicitly approved it.
7. Discuss with students: this is exactly how real deployment pipelines protect production systems — automated stages (checkout, build, test) run freely, but the moment something risky or irreversible is about to happen (like deploying to real users), a human checkpoint is inserted on purpose.

PROJECT 4: Environment Variables + Linting Pipeline

Concept taught: Jenkins auto-injects built-in environment variables (`BUILD_NUMBER`, `JOB_NAME`, `WORKSPACE`) into every run.

Files to create in the GitHub repo

`app.py`

```
def greet(name):  
    return "Hello, " + name
```

`Jenkinsfile`

```
pipeline {  
    agent any  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'  
            }  
        }  
  
        stage('Show Build Info') {  
            steps {  
                echo "Build Number: ${env.BUILD_NUMBER}"  
                echo "Job Name: ${env.JOB_NAME}"  
                echo "Workspace: ${env.WORKSPACE}"  
            }  
        }  
  
        stage('Run Linter') {  
            steps {  
                bat 'flake8 app.py'  
            }  
        }  
    }  
}
```

Execution procedure

1. Create a new GitHub repo, add `app.py` and `Jenkinsfile`, push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now**, then click it again (**Build Now** a second time).

4. Open **Console Output** for both builds and compare — BUILD_NUMBER increases (e.g. #1 then #2) while JOB_NAME and WORKSPACE stay the same. This shows Jenkins tracks build identity automatically.
5. Since app.py above is clean, “Run Linter” should pass. **Try it broken:** add a line with a trailing space or an unused import, push, rebuild — show that flake8 now fails the stage.

PROJECT 5: Post-Build Success/Failure Pipeline

Concept taught: the `post` block — pipelines can react differently depending on whether the build passed or failed. This is separate from the numbered `stages`.

Files to create in the GitHub repo

app.py (a clean, compilable file)

```
def multiply(a, b):  
    return a * b  
  
print(multiply(4, 5))
```

Jenkinsfile

```
pipeline {  
    agent any  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/<student-username>/<repo-name>.git'  
            }  
        }  
  
        stage('Compile Check') {  
            steps {  
                bat 'python -m py_compile app.py'  
            }  
        }  
    }  
  
    post {  
        success {  
            echo 'Build succeeded: app.py has no syntax errors.'  
        }  
        failure {  
            echo 'Build failed: check app.py for syntax errors.'  
        }  
    }  
}
```

Execution procedure

1. Create a new GitHub repo, add app.py and Jenkinsfile, push.
2. In Jenkins, create a new Pipeline job pointing to this repo.
3. Click **Build Now** — confirm the build is marked **success** (green) and the console shows the “Build succeeded” message from the `post` block.

4. **Try it broken:** introduce a syntax error in `app.py` (e.g. remove a closing parenthesis: `print(multiply(4, 5))`), push, and rebuild.
5. Confirm the build is marked **failed** (red) and the console shows the “Build failed” message instead — demonstrating that `post` runs regardless of outcome, but chooses the block matching what happened.

Summary table

	Concept	Stage 2	Stage 3 / post
Project 1	Automated testing (basics)	Install dependencies	Run unit tests
Project 2	Parametrized tests (multiple cases, one test function)	Install dependencies	Run unit tests (verbose)
Project 3	Manual approval gate before deploy	Build (compile check)	Deploy (with <code>input</code> approval)
Project 4	Built-in environment variables	Show build info	Run linter
Project 5	Post-build success/failure hooks	Compile check	<code>post { success / failure }</code>

Note on Project 5: it has only 2 stages inside the `stages` block, plus a `post` block — worth explicitly telling students that `post` is a lifecycle hook, not a numbered stage.